

Distributed Systems - Notes - v0.1.0

260236

October 2026

Preface

Every theory section in these notes has been taken from the sources:

- Course slides. [1]

About:

 [GitHub repository](#)



These notes are an unofficial resource and shouldn't replace the course material or any other book on distributed systems. It is not made for commercial purposes. I've made the following notes to help me improve my knowledge and maybe it can be helpful for everyone.

As I have highlighted, a student should choose the teacher's material or a book on the topic. These notes can only be a helpful material.

Contents

1	Introduction	4
1.1	Why Distributed Systems?	4
1.1.1	Motivation: Why Distributed Systems?	4
1.1.2	Why Distribute a System?	14
1.1.3	Why Distributed Systems Are Special	18
1.2	What is a Distributed System?	23
1.3	Middleware	28
1.4	Fundamental Properties	31
	Index	35

1 Introduction

1.1 Why Distributed Systems?

1.1.1 Motivation: Why Distributed Systems?

There is a very simple reason why distributed systems are important: most applications we use today are **distributed**. Many applications cannot realistically run as a single program on a single computer.

Example 1: Distributed Applications

E-mail, web/cloud applications such as *Google Maps*, *Google Docs*, *Microsoft Office 365*; mobile applications and mobile devices, web services and websites, and many more, are all distributed systems.

E-mail is used as an especially clear example because sending a message involves several independent machines communicating over a network:

User A → mail server A → network → mail server B → User B

There is no single computer performing the entire operation.

So the first intuition is that **modern applications are naturally composed of multiple communicating machines**. This is why distributed systems are not some niche branch of computer science, they are behind a huge fraction of modern computing infrastructure.

State of the Internet

Censys on 2023 created a report about the **observable public Internet**, built from large-scale Internet scanning.¹ The report says that in one daily snapshot in early 2023, Censys observed more than **740 million hosts** exposing about **1.3 billion HTTP services**, roughly **18%** hosted by major cloud providers such as Amazon, Oracle, Google Cloud, or Azure. And more than **85%** of HTTP services running on named hosts.

 A distributed system may therefore consist not of two or three machines, but potentially of 10^3 , 10^5 , 10^6 , or even more interacting components. But this immediately hints at problems that will appear later in the notes: scalability, failures, communication, naming, synchronization, replication, and so on. But at this point, we are just trying to understand why distributed systems are important.

Large-scale Web traffic

From the same Censys report, we can also see the **traffic** perspective rather than the number of machines. For example, the report says that in 2023, on Google:

¹Censys is a platform that **scans and indexes Internet-connected systems and services**. It is like a search engine for Internet infrastructure.

- More than **100k searches per second**
- More than **100 billion visits per month**
- YouTube as another extremely large service
- Google and YouTube producing more traffic than many other major websites combined

Again, the exact values are secondary. The important question is: *could a single server handle all of this?* Clearly not. A large Web service therefore has to distribute work across many machines. And this is the first glimpse of the **scalability** problem that distributed systems have to solve.

In summary, **modern computing operates at a scale where distribution is not just a design choice, but unavoidable.** We saw how widespread distributed applications are, the size of the Internet, and finally the traffic generated by huge Web services.

🌲 LLM infrastructures as distributed systems

Let's consider what happens when we send a query to a modern Large Language Model (LLM) such as ChatGPT. It is tempting to imagine:

Our request → one very powerful GPU → our answer

But this is not how it works. Modern LLMs are too large to fit on a single GPU. So a single query to a modern LLM may involve roughly **16 to 32 GPUs**, corresponding to a couple of multi-GPU nodes. Each GPU may consume up to about **700 W**.

So even a single logical request may require computation spread across several physical processors:

$$\text{Request} \rightarrow \begin{cases} \text{GPU}_1 \\ \text{GPU}_2 \\ \text{GPU}_3 \\ \vdots \\ \text{GPU}_{16-32} \end{cases} \rightarrow \text{Answer}$$

From the user's perspective, however, we do not see any of this. We simply **interact with one service: LLM service.** This is already one of the central ideas of distributed systems: **many independent computational resources cooperate to provide what appears to the user as one single service.** The infrastructure becomes even more clearly distributed when we move from one request to the whole data center. An AI data center may contain **more than 100,000 AI accelerators**, with power consumption potentially reaching hundreds of megawatts.

🧩 So there are actually **two different forms of distribution** here:

- At the level of a **single request**:

One Computation $\rightarrow$ Multiple GPUs

- At the level of the **service as a whole**:

Millions of requests $\rightarrow$ Many machines $\rightarrow$ Many clusters
 $\rightarrow$ Possibly many data centers

The reason this example matters is that distribution is not only about connecting independent applications. Sometimes even **one computation is too large to execute efficiently on one processor**. That naturally connects distributed systems with high-performance computing (HPC), although the two are not identical. However, **modern AI services are distributed both because of computational size and service scale**.

Google as a distributed “supercomputer”

Eric Schmidt (CEO of Google from 2001 to 2011) described Google as essentially “*a large distributed supercomputer spread across many data centers*”. That is a very useful mental model. Instead of imaging Google as a single website, we can think of it as an **enormous collection of cooperating computers** spread geographically across the world.

Data center A $\leftrightarrow$ Data center B $\leftrightarrow$ Data center C $\leftrightarrow$...

And inside each data center, there are many cooperating machines:

server₁, server₂, server₃, ...

All connected through networking infrastructure.

Google historically operated hundreds of thousands of servers and gives a more recent estimate of **more than 2 million servers across 36 data centers**. The important point is not the exact count; it is the architectural idea. From outside, we might type `google.com` and perceive a single service. But internally, that service may involve an enormous distributed infrastructure. So we get an important contrast:

- **User’s view**: Google, a single website.
- **Physical reality**: millions of machines, networks, and data centers cooperating to provide a single service.

This is a recurring objective in distributed systems: **expose a coherent service while hiding much of the underlying distribution**. We will later give this idea a precise name: **transparency**.

 There is also another important implication. If the system contains millions of machines, failures cannot be treated as exceptional events. Suppose each individual machine is highly reliable. With millions of components, some machine

somewhere will still fail regularly. So at very large scale, **failure** is not something that might happen once in a while. It **becomes something the system must be designed to live with**. This point will become central when we discuss **partial failures** and **fault tolerance** later. For now, the main takeaway is that **a huge distributed infrastructure can behave externally like one computer/service**.

Distributed systems beyond the Internet

At this point it would be easy to conclude that distributed systems are only internet applications. That would be **wrong**. The Internet, the Web, and LLM platforms are distributed systems, but many distributed systems have little or nothing to do with the public Internet.

Example 2: Distributed Systems Beyond the Internet

- **Intranets**. An organization can have a private distributed system. For example, banking systems, corporate systems, scientific computing infrastructure, server farms, grids, clouds.
For example, imagine a bank with branches in Milan, Rome, and Turin. The system is distributed even if its internal communication is largely private and not exposed to the public Internet.
- **Home networks**. Even our home can contain a small distributed environment: laptop, router, tv, NAS, phone, and so on. Different devices communicate and share resources. This is a distributed system on a dramatically smaller scale than Google, but the idea is the same: independent devices and communication.
- **Personal Area Network (PAN)**. A person's devices can themselves form a small network: phone, smartwatch, headphones, gps. For example healthcare-monitoring devices can communicate with a phone and send data to a cloud service. Again, **distribution does not imply thousands of servers. Two or three cooperating devices are already enough to introduce distributed-system concepts**.
- **Wireless Sensor Networks**. Another important category consists of many small sensing devices communicating with one another or with gateways. For example, a smart home may have many sensors and actuators communicating with a central hub. Or a smart city may have thousands of sensors monitoring traffic, pollution, or infrastructure health. These are distributed systems even if they are not part of the public Internet.

Thus a distributed system may exist across continents, inside one company, inside one building, inside one house, or even around one person. What matters is not the geographical size, but that **multiple independent computational components communicate and cooperate**.

🏠 Home and personal networks

Until now, we have mostly thought about distribution at a very large scale: Google, cloud infrastructures, LLM data centers. But a distributed system can also exist on a much smaller scale.

For example, inside a home we might conceptually have a PC, a laptop, a media device, connected through a local network and possibly communicating with remote machines through the Internet. So **distribution is not defined by physical distance**. Two computers sitting in the same room can form part of a distributed system just as computers located on different continents can.

In the previous page, we mentioned home networks and Personal Area Networks (PANs). A home network concerns devices belonging to the environment:

$$\text{home} \rightarrow \{\text{computers, entertainment devices, shared resources}\}$$

A personal-area network concerns devices surrounding a person:

$$\text{person} \rightarrow \{\text{phone, smartwatch, headphones, gps, etc.}\}$$

⚠️ The scale is small, but the distributed-system issue remains: **independent components must communicate and coordinate**.

🏠 Pervasiveness of Distributed Systems

Instead of having a few clearly recognizable computers (desktop, laptop, server), **computing and communication capabilities become embedded in ordinary physical objects**. This is the basic idea behind the **Internet of Things (IoT)**. We can use three quotations to develop this idea:

1. **Computing that “disappears”**. Mark Weiser (1991) says that the most profound **technologies** eventually become **integrated into everyday life** to the point that we **stop consciously noticing them**. This is the idea of **Ubiquitous Computing** (or **Pervasive Computing**).

Think about the conceptual transition from “*use a computer*” toward “*computing is embedded in the environment around us*”. The computer itself is no longer necessarily the object we interact with.

2. **Connectivity for things**. The ITU (International Telecommunication Union) quotation captures another transition. Traditional networking gave us connectivity from anyone to anywhere (i.e., from one person to another). IoT extends this toward **connectivity for anything**. Objects that previously had no communication capabilities can contain small transceivers and participate in a network. So instead of only connecting computers, we may have a network of connected objects. The number of communicating entities consequently becomes enormous, **21.1 billion active IoT connections by the end of 2025**.
3. **From connected objects to intelligent objects**. The Negroponte quotation adds another important point. IoT is not merely attaching an

identifier to an object, such as an RFID tag that tells us where the object is. IoT is about **embedding intelligence into objects**. So a device can potentially:

- (a) Observe something about its environment (e.g., temperature, humidity, motion, light, etc.)
- (b) Process information (e.g., detect patterns, make decisions, etc.)
- (c) Communicate with other devices (e.g., send data, receive commands, etc.)
- (d) Participate in a larger system (e.g., a smart home, a smart city, etc.)

This gives us a useful progression: from ordinary objects, to connected objects, to **networked computational components**. And once we have potentially billions of these components, distributed-system techniques become essential.

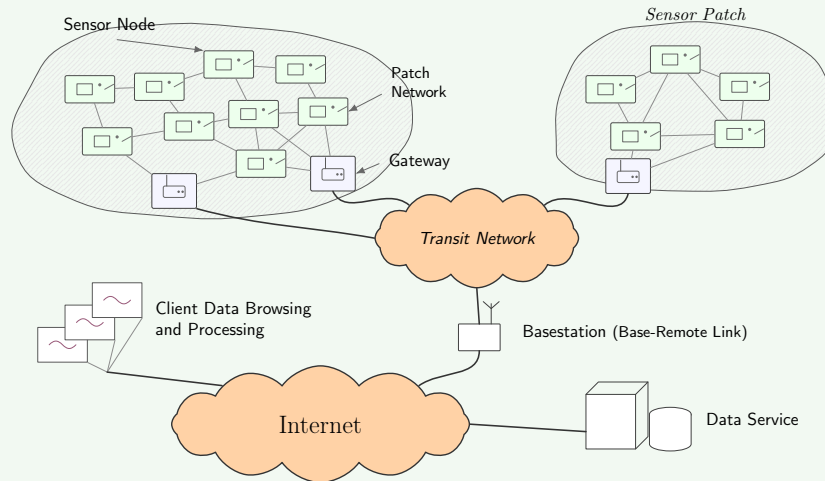
Example 3: Why a Wireless Sensor Network is a pervasive system

Imagine we want to monitor a foster, a factory, a building, or a crop field. Instead of putting one big computer in the middle, we scatter many tiny devices around the environment:

$$S_1, S_2, S_3, \dots, S_n$$

Each one may measure something like: temperature, humidity, light, vibration, pressure, motion, etc. These devices are the **sensor nodes** and they are placed on the edge of the environment.

The crucial idea is that computing becomes part of the environment itself. We are no longer just using a computer to monitor the environment. Instead, the sensors are just “there”, continuously observing the physical world. That is exactly why this is **pervasive computing**: computation is distributed throughout the environment, and it is not necessarily associated with a single device that we consciously interact with.



In the previous diagram, we represent a generic architecture for a Wireless Sensor Network (WSN).

- On the top-left, we have small devices inside the shaded region, called **sensor nodes**. Each one is a small computer with, conceptually, a CPU, memory, and a small radio transceiver. The sensor observes something physical, for example temperature, humidity, light, vibration, pressure, motion, etc.
- A group of sensor nodes may form a **Sensor Patch** (top-right), which is a small area of the environment that is monitored by a group of sensors. Inside each patch, the sensor nodes communicate through the **Patch Network**. Therefore, the sensors form a local network rather than each sensor independently connecting directly to the Internet.
- At the edge of each Sensor Patch, we have a **Gateway**. The sensor network may use some specialized local wireless technology, while the rest of the system uses a different network. The gateway bridges the two networks. **So instead of each sensor node connecting directly to the Internet, the gateway collects data from the sensor nodes and sends it to the Internet.**
- Multiple gateways connect to the **Transit Network**. This part simply transports information from the local sensor patches toward a more conventional infrastructure.
- The transit network reaches a **Base Station**. This is the point where data can be forwarded toward a larger network infrastructure. We call it also a **Base-Remote Link**, because it connects the local sensor network to a remote infrastructure.
- Once the information reaches the Internet, it can be sent to the **Data Service** shown on the bottom-right. This might conceptually be a database, a server, a cloud service, or an analytic system.

For example, suppose the sensors are measuring temperature. The data service could receive: $(t_1, 22.4^\circ C)$, $(t_2, 22.5^\circ C)$, $(t_3, 22.6^\circ C)$, and store or process those values.

- Finally, on the left of the Internet cloud, we have **Client Data Browsing and Processing**. That is simply the user/application consuming the collected data.

For example, a control software could receive the temperature data and decide to turn on a fan or an air conditioner if the temperature exceeds a certain threshold.

This example shows that distributed systems are not just servers communicating over the Internet. They can also be pervasive systems, where computation is embedded in the environment itself.

❓ Why mobility makes distributed systems harder

We can consider three different mobility scenarios that make distributed systems more challenging:

- **Normadic**. In the nomadic case, the device moves between locations, but communication is associated with whichever fixed network access point it is currently using.

For example, a laptop may move between different Wi-Fi access points in a building. Thus, the **device changes its point of attachment** to the network, and the natural question is “*how do we continue referring to the same device when its network location changes?*”. This question will become important later when we discuss *naming*.

- **Base-station mobility**. Think about a mobile phone on a cellular network. We are moving, but there is a fixed infrastructure around us. Suppose we are walking through Milan while on a call. At first our phone communicates through BS_1 , then we move far enough that another base station becomes better, so we switch to BS_2 .

The important thing is that the **device moves, but the infrastructure does not**. The network has fixed access points/base stations, and the mobile node attaches to whichever one is appropriate. The distributed-system challenge is that the device’s **point of attachment changes while the device itself remains the same entity**. So again, identity and current location must be kept separate.

- **Ad-hoc mobility**. Imagine several laptops or phones in the same area, and they can communicate directly with one another through their wireless interfaces.

$$A \leftrightarrow B \leftrightarrow C$$

If A cannot directly reach C , but B can reach both, then B may conceptually act as an intermediate node. The important feature is that **the devices themselves form the network**. There is no fixed base station organizing the connectivity.

For example, imagine several rescue teams entering an area where there is no working network infrastructure. Their devices may form a temporary network among themselves:

$$A \leftrightarrow B \leftrightarrow C \leftrightarrow D$$

Then people move. Maybe B leaves the area, so the topology changes:

$$A \leftrightarrow C \leftrightarrow D$$

Or perhaps A can no longer reach the others at all. That is why ad-hoc networks are especially dynamic: the **network topology itself changes as the nodes move**.

The three cases can be summarized as follows:

Model	Device moves?	Fixed infrastructure?
Nomadic	Yes, between attachment points.	Yes
Base-Station	Yes	Yes, explicitly through base stations.
Ad-Hoc	Yes (potentially)	No, fixed base station.

Even without mobility, a distributed system already needs to know:

- *Who is the other component?* (identity)
- *How can I reach it?* (location)

With mobility, the **second answer may continuously change**.

✓ Key Takeaways 1: Why Distributed Systems?

- Most modern applications are inherently **distributed**: their computation, data, or users span multiple communicating machines.
- Distribution is often unavoidable because of **scale**. Services such as large Web platforms and LLM infrastructures cannot realistically be provided by a single machine.
- A distributed system may involve anything from a few personal devices to millions of servers. Therefore, **distribution is not defined by geographical distance or system size**.
- Large infrastructures illustrate an important recurring idea:

Many independent components $\longrightarrow$ one coherent service

The user may perceive a single service even though the implementation is spread across many machines. This anticipates the concept of **transparency**.

- At sufficiently large scale, **failures are normal rather than exceptional**. A distributed system must therefore be designed assuming that individual components may fail while the rest of the system continues operating.
- Distributed systems are not limited to the Internet or the Web. Examples include: intranets, home networks, personal area networks (PANs), Internet of Things (IoT) systems, and wireless sensor networks (WSNs).
- **Pervasive/ubiquitous computing** pushes computation into ordinary objects and environments. IoT extends connectivity from people and computers toward potentially huge numbers of **networked intelligent objects**.
- A **Wireless Sensor Network** is a representative pervasive distributed system: sensor nodes form a local patch network, which connects to a gateway, which connects to a transit network, which connects to the Internet or a data service.

The gateway bridges the local sensor network with the larger communication infrastructure.

- **Mobility makes distribution harder** because the identity of an entity and its current network location must be separated:
 - **Nomadic**: the device moves between different attachment points.
 - **Base-station**: the mobile device communicates through a fixed infrastructure and may switch from one base station to another.
 - **Ad-hoc**: there is no fixed infrastructure; the mobile nodes themselves form the network, so the network topology may change as they move.

1.1.2 Why Distribute a System?

Economics

Historically, computing was often organized around large centralized machines, such as **mainframes** (i.e., a single computer that serves multiple users), with many users or terminals depending on that central machine. The alternative is to use many smaller machines $M_1, M_2, M_3, \dots$ and let them cooperate. The economic argument is based on the **price/performance ratio**.

Instead of buying one extremely powerful machine, it **may be more convenient to buy several cheaper machines and combine their resources**. The idea is not that several small computers automatically behave like one large computer. We need software that coordinates them. But **distribution allows us to exploit the aggregate resources of many machines**. So the economic motivation is that **it can be cheaper to obtain computing power by combining multiple machines**. This is one of the historical reasons why distributed computing became attractive.

Incremental growth and load balancing

This is one of the most important practical advantages. Suppose a centralized system starts with one machine M_1 and its workload increases. Eventually:

$$\text{load} > \text{capacity of } M_1$$

With a distributed architecture, we can add more machines:

$$M_1 \rightarrow M_1 + M_2 \rightarrow M_1 + M_2 + M_3 \rightarrow \dots$$

This is **incremental growth**. We do not necessarily need to replace the entire system with a much larger machine every time demand increases. Instead, **capacity can grow by adding resources**. This is closely related to **load balancing**.

Imagine that three machines are available:

$$M_1, M_2, M_3$$

And requests arrive:

$$R_1, R_2, R_3, R_4, R_5, R_6$$

Rather than sending everything to M_1 :

$$R_1, R_2, R_3, R_4, R_5, R_6 \rightarrow M_1$$

We can distribute them:

$$R_1, R_2 \rightarrow M_1, \quad R_3, R_4 \rightarrow M_2, \quad R_5, R_6 \rightarrow M_3$$

The objective is to use the available resources more uniformly.

So there are really two connected ideas:

- **Incremental growth:** add resources gradually as the system grows.
- **Load balancing:** spread the workload among the available resources.

In summary, it makes easier to **evolve the system** and **use its resources uniformly**.

Inherent distribution

Sometimes we distribute a system because we want the benefits of distribution. But in other cases, **distribution is not really a design choice, but rather the problem itself is already distributed.**

For example, consider a bank. Its users, branches, ATMs, databases, and services may naturally exist in different locations: ATM Milan, ATM Rome, ATM New York, etc. All of them interact with the same overall banking system. Trying to force everything into one physical place would not make sense (i.e., it would be inefficient and impractical). This is what **inherent distribution** means: **the entities involved in the problem are naturally located in different places.**

Reliability

Distribution can also improve **reliability**. Consider a purely centralized architecture client-server. If that server fails, the entire service may disappear. This is a **single point of failure**. Now suppose the system is distributed over several components $S_1, S_2, S_3, \dots$. If one of them fails, the others may still be able to provide the service. So **one failure does not necessarily bring down the whole system.** This is particularly important for **mission-critical applications**, where complete service failure is unacceptable.

But there is an important subtlety. Distribution does **not automatically guarantee reliability**. It merely gives us the possibility of designing a system that survives individual failures. For example, if all machines depend on one central database, then the database may still be a single point of failure. Thus, to obtain reliability, we often need techniques such as **redundancy, replication, failure detection, and recovery**. These are all topics that will be covered in later sections. However, here the main point is that **distribution allows failures to be partial rather than necessarily total.** That sounds good, and it often is, but partial failures will also turn out to be one of the hardest aspects of distributed systems.

The complexity cost of distribution

Obviously, distribution is not free. **Distribution systems are considerably more difficult to get right.** Everything we have discussed sounds attractive: lower cost, easy growth, load balancing, reliability, and so on. But none of this is free.

Consider two components A and B on two different machines. If A needs something from B , communication now happens through a network:

$$A \xrightarrow{\text{network}} B$$

That introduces many questions:

- *What if the message is delayed?*

$$A \xrightarrow{\text{delayed}} B$$

- *What if the message is lost?*

$$A \not\xrightarrow{\text{lost}} B$$

- *What if B has crashed?*
- *What if B is alive but just very slow?*

From A 's perspective, slow network and failed machine may be difficult to distinguish. Then there are **synchronization problems**.

Suppose two machines modify the same logical data:

$$A : x \leftarrow 5$$

$$B : x \leftarrow 7$$

Which value should the system consider correct? There is also no single perfectly shared notion of time:

$$\text{clock}_A \neq \text{clock}_B$$

And components may fail independently.

So distributing a system trades one kind of difficulty for another. A centralized system may be easier to reason about one machine at a time, while a distributed system gives us additional capabilities (many machines) but also introduces new problems: communication, concurrency, partial failure, coordination, and so on. This is the central trade-off.

✓ Key Takeaways 2: Why Distribute a System?

There are four main motivations for distributing a system:

1. **Economics.** Several smaller machines may provide a better **price/performance ratio** than one extremely powerful centralized machine.

many cheaper machines $\longrightarrow$ aggregate computational resources

2. **Incremental growth and load balancing.** A distributed system can grow gradually by adding resources as the workload increases:

$$M_1 \rightarrow M_1 + M_2 \rightarrow M_1 + M_2 + M_3 \rightarrow \dots$$

The workload can then be distributed among those resources rather than concentrating it on a single machine.

3. **Inherent distribution.** Sometimes the problem itself is naturally distributed: users, resources, or activities already exist at different locations.

distributed problem $\implies$ naturally distributed solution

Typical examples are banking systems, reservation systems, distributed games, collaborative applications, and mobile applications.

4. **Reliability.** Distribution makes it possible for the system to continue operating even if one component fails:

failure of one component $\nRightarrow$ failure of the entire system.

This is especially important for **mission-critical systems**. However, reliability is not automatic: the architecture must avoid introducing new single points of failure.

So in general, **distribution brings advantages, but it also makes systems harder to build correctly.** Communication, coordination, concurrency, and failures introduce complexity that does not appear in the same form in centralized systems.

1.1.3 Why Distributed Systems Are Special

🔗 Distributed vs. Centralized Systems

Start from the simplest possible centralized program: an **application running on a single machine**. Its components share the same machine, operating system, memory hierarchy, and physical environment. If one part of the program invokes another function, conceptually we have:

$$A \xrightarrow{\text{function call}} B$$

And both A and B are local (i.e., on the same machine).

Now **distribute** those components A and B with a network between them:

$$A \xrightarrow{\text{network}} B$$

That apparently small change is **enormously significant**. A local interaction may involve something like a function call, while a distributed interaction requires:

1. Serialization of data into a message
2. Communication over a network
3. Network transmission, which may be delayed or lost
4. Reception of the message by B
5. Deserialization of the message into data
6. Processing of the data by B

And every one of those steps introduces additional uncertainty and potential failure. This is why **a distributed system is not simply a centralized program running on several machines**. The network becomes part of the computation.

❓ Why building distributed systems is harder

As we saw in the previous comparison, distributed systems are more complex than centralized systems. Not only is **building them harder**, but **building them correctly is much harder**.

In general, it is often relatively easy to make a distributed application work when everything behaves normally. For example:

$$A \rightarrow B, \quad B \rightarrow A$$

Request arrives, reply arrives, everything is fine. The **difficulty** appears when reality interferes. Suppose A sends a request to B :

$$\text{transfer}(100 \text{ €})$$

B receives the request and performs the operation. Then B sends a reply back to A , but it is lost in the network. *What should A do?*

- If it sends the request again, perhaps the transfer happens **twice**.
- If it does not retry, perhaps the first request had actually been lost and the transfer happens **zero times**.

So even something that sounds trivial, like “*retry when there is an error*”, may not be trivial at all. The **problem is that different machines have only partial knowledge of what happened**.

☰ The Eight Fallacies of Distributed Computing

The **Eight Fallacies of Distributed Computing**, associated with Peter Deutsch, are **assumptions that programmers are tempted to make when first designing distributed software**:

1. The network is reliable
2. Latency is zero
3. Bandwidth is infinite
4. The network is secure
5. Topology doesn't change
6. There is one administrator
7. Transport cost is zero
8. The network is homogeneous

They are called “*fallacies*” because **none of these assumptions is generally safe**. Let's analyze each of them in turn:

1. **The network is reliable**. A common mental model is:

$$A \xrightarrow{\text{message}} B$$

Therefore B receives the message. But in reality, this is not necessarily true. Packets can be lost, corrupted, delayed, duplicated, or delivered after a connection failure. So the actual model is closer to:

$$A \xrightarrow[\text{maybe}]{\text{network}} B$$

This means **distributed software often needs mechanisms such as acknowledgments, retries, duplicate detection, or replication**. However, the important principle is:

$$\text{sending} \neq \text{receiving}$$

2. **Latency is zero**. A local memory access or function call is extremely fast compared with network communication. If A communicates with B :

$$A \xrightarrow{\Delta t}$$

Where Δt is the time it takes for the message to travel from A to B , which is $\Delta t > 0$, and it may vary substantially.

A remote operation can therefore never safely be treated exactly like an ordinary local operation. For example, imagine writing:

```

1 for each item:
2     remoteServer.get(item)

```

If every operation requires a network round-trip, latency accumulates:

$$T \approx n \cdot RTT$$

Where T is the total time, n is the number of items, and RTT is the round-trip time (i.e., the time it takes for a message to go from A to B and back). So architecture and algorithms must take communication latency seriously.

3. **Bandwidth is infinite.** A network can move only a finite amount of data per unit time:

$$\text{bandwidth} < \infty$$

If many machines exchange large amounts of information, the network itself may become the bottleneck.

Imagine 1000 machines constantly sending gigabytes of state to each other. Even if every CPU is mostly idle, the system can still perform badly because the **network capacity is a resource**. This is one reason distributed algorithms often try to reduce communication.

4. **The network is secure.** Once communication crosses a network, additional attack surfaces appear. Data may potentially be intercepted, modified, or an attacker may impersonate another component.

So communication may require authentication, encryption, integrity protection, authorization. This fallacy connects directly with the security challenge that will appear later in the notes.

5. **Topology doesn't change.** Suppose our system contains: A , B , C , and D connected in a line. It is easy to design an algorithm assuming that this arrangement will remain stable. But **distributed systems change**. A node can **fail**:

$$A - B - \emptyset - D$$

A new node can join:

$$A - B - C - D - E$$

A mobile device can move, a router can disappear, or a service can migrate. Therefore the **set of nodes and connections is dynamic**. This becomes especially obvious in the mobile and ad-hoc systems we saw in the previous section (page 11).

6. **There is one administrator.** On our own computer, one administrative authority can generally configure the whole system. Instead, large distributed systems may cross organizations, departments, companies, cloud providers, countries. **There may be no single administrator who controls everything**. Different domains may have different policies, security rules, configurations, software versions. So distributed systems often have to **operate across administrative boundaries**.

7. **Transport cost is zero.** **Communication has a cost.** That cost can include:

time + bandwidth + CPU + energy + possibly money

Consider two implementations.

- **Version A.** Send 1000 remote messages:

$1000 \times \text{network operation}$

- **Version B.** Aggregate the information and send one message:

$1 \times \text{network operation}$

Even if they compute logically the **same result**, their **performance** can be **dramatically different**. So **communication is not free**. This idea will appear repeatedly throughout distributed systems.

8. **The network is homogeneous.** **A distributed system may combine machines with completely different characteristics:** x86 server, ARM phone, IoT sensor, cloud VM, using different hardware, operating systems, network technologies, data representations, programming languages. Therefore **distributed systems are inherently heterogeneous**.

❓ **What the Eight Fallacies of Distributed Computing mean for us?**
We should not memorize them merely as 8 sentences. They are really **8 warnings** against pretending that:

remote $\equiv$ local

A remote component differs fundamentally from a local one because the **network lies between us and it**. So a good distributed-system design starts from almost the opposite attitude: **assume that the network is unreliable, slow, and costly**. And then **builds guarantees on top of that unreliable and changing environment**.

📌 Key Takeaways 3: Why Distributed Systems Are Special

A distributed system is fundamentally different from a centralized one because components communicate through a **network**:

local interaction $\neq$ remote interaction

A network introduces delay, limited capacity, failures, changing topology, security concerns, and heterogeneity. Consequently, distributed systems are:

more complex to design, harder to build, and much harder to build correctly.

The Eight Fallacies of Distributed Computing summarize the assumptions that must **not** be made when designing a distributed system:

1. The network is reliable.
2. Latency is zero.
3. Bandwidth is infinite.
4. The network is secure.
5. Topology does not change.
6. There is one administrator.
7. Transport cost is zero.
8. The network is homogeneous.

They can be interpreted as four broad warnings:

- **Communication is uncertain.** Sending a message does not imply that it is received, and delays may be unpredictable.
- **Communication is not free.** Remote operations have latency, consume bandwidth, and require additional processing and resources.
- **The environment changes.** Nodes, links, locations, administrators, and network topology may change over time.
- **Distributed systems are heterogeneous.** Components may use different hardware, operating systems, networks, protocols, and software environments.

Therefore, a fundamental distributed-systems principle is:

do not design a remote interaction as if it were local

Correct distributed software must explicitly deal with communication, concurrency, uncertainty, and failures instead of assuming them away.

1.2 What is a Distributed System?

There is no single definition of a distributed system. Here we provide **three different definitions**, each of which emphasizes a different aspect of distribution.

Definition 1: Tanenbaum & Van Steen – The user’s perspective

A **Distributed System** is a collection of independent computers that appears to its users as a single coherent system.

In the first definition, Tanenbaum & Van Steen emphasize two parts, and **both matter**:

- “*Collection of independent computers*”. The **system** is physically composed of multiple autonomous machines:

$$C_1, C_2, \dots, C_n$$

Each machine has its own processor, memory, operating system, and can potentially fail independently.

- “*Appears to its users as a single coherent system*”. Externally we want:

$$C_1 + C_2 + \dots + C_n \approx \text{one coherent system}$$

The user should not necessarily have to think: “*my request is currently being handled by machine 37 in data center B*”. Instead, the user sees something like *Google Search* or *online banking service* even though internally many machines cooperate.

So this definition combines two seemingly contradictory ideas:

physical multiplicity and logical unity

In other words, **many machines underneath, one system from the outside**. It anticipates the concept of **transparency**, which we will see later.

Definition 2: Lamport – The failure perspective

A **Distributed System** is a system on which I cannot get any work done because some machine I have never heard of has crashed.

It sounds like a joke, but it highlights one of the **deepest properties** of distributed systems. Imagine that we are using service *A*. From our point of view, perhaps *A* is working perfectly. But internally:

$$A \rightarrow B \rightarrow C \rightarrow D$$

And A depends on B , which depends on C , and so on. Now suppose that C crashes. Then B cannot complete its work, and A cannot complete its work either:

$$C \text{ fails} \implies B \text{ cannot proceed} \implies A \text{ cannot serve users}$$

So our application fails because of a remote component completely invisible to us. This reveals an essential property: **components fail independently**. A centralized system often has a simpler failure model: if the central server fails, the whole system fails. In a distributed system, failures can be more subtle and complex:

$$C_1 : \checkmark \quad C_2 : \checkmark \quad C_3 : \times \quad C_4 : \checkmark$$

The system is neither completely alive nor completely dead. This is what we later call **apartial failure**. Therefore, Lamport's definition emphasizes that **distribution creates dependencies on independently failing components**.

Definition 3: Coulouris – The communication perspective

A **Distributed System** is a system in which hardware or software components located at networked computers communicate and coordinate their actions only by passing messages.

Suppose we have two processes P_1 and P_2 running on different machines. They do **not** communicate by directly reading and writing the same physical memory. Instead, they communicate by sending messages over the network:

$$P_1 \xleftrightarrow{\text{network}} P_2$$

Communication and coordination occur through the network. So if P_1 wants information held by P_2 , it cannot simply do:

$$x = \text{memory}[\text{address}]$$

It needs some form of communication:

$$\text{request} \rightarrow \text{network} \rightarrow P_2$$

Followed potentially by:

$$P_2 \rightarrow \text{network} \rightarrow \text{reply}$$

This immediately explains why the network matters so much. However, the network introduces a new source of complexity: delay, message loss, finite bandwidth, failures, uncertainty, and so on. Therefore, **message passing is not merely an implementation detail**, it fundamentally changes how programs must be designed.

Therefore, the 3 definitions are not competing definitions, they describe the same object from different perspectives:

- **Structural viewpoint.** Multiple independent computers that should appear as one coherent system.
- **Failure viewpoint.** Independent components can fail and affect one another.
- **Communication viewpoint.** Components coordinate through message passing.

Together, we obtain a very useful working definition.

Definition 4: Distributed Systems

A **Distributed System** consists of independent networked components that coordinate through message passing to provide some coherent over-all service.

 **Independent does not mean isolated.** When we say “*independent networked components*”, we do **not** mean that C_1 and C_2 are completely isolated with no interaction.

If they were completely isolated, we would simply have two unrelated computers. Instead, we mean that they are **autonomous**, in the sense that each component has its own processor, memory, and operating system, but they can **cooperate** to provide a service.

Parallel Computing vs. Distributed Computing

A natural question is: *what is the difference between parallel computing and distributed computing?* At first sight, the two fields look almost identical.

- **Parallel computing** $P_1, P_2, \dots, P_n$ perform computation together.
- **Distributed computing** $C_1, C_2, \dots, C_n$ also perform computation together.

So why distinguish them? Because the **assumptions about the architecture are different**.

 **How tightly coupled are the processors?** A useful way of understanding the distinction is through **memory and communication**. Consider two processors CPU₁ and CPU₂:

- **Shared-Memory Parallel System.** The two processors may **share the same shared memory**. Then both processors can access variables stored in that memory.

If $x = 5$ is stored in shared memory, both processors can potentially read x . Communication can therefore happen implicitly through memory:

$$\text{CPU}_1 : x \leftarrow 5 \quad \text{and} \quad \text{CPU}_2 : \text{read } x$$

This is characteristic of a **tightly coupled** parallel machine.

- **Distributed Systems use Private Memories.** Now consider two separate computers:

$$\text{CPU}_1 + \text{Memory}_1 \quad \text{and} \quad \text{CPU}_2 + \text{Memory}_2$$

There is no physical shared memory between them. Communication can only happen through message passing over the network:

$$\text{CPU}_1 \xleftrightarrow{\text{network}} \text{CPU}_2$$

So the conceptual difference is that **parallel computing assumes shared memory, while distributed computing assumes private memories but networked communication**. This is exactly why the Coulouris definition stresses the sentence: “*only by passing messages*”.

Furthermore, there is another architectural dimension: **how processors are interconnected**. A simple multiprocessor may use a common bus to connect all processors with shared memory. This makes communication very direct, but a common bus does not scale indefinitely because all processors compete for the same interconnect.

⚠ More sophisticated parallel machines may therefore use switching networks connecting processors and memories. Thus, *parallel computing is similar to distributed computing, right?* Not quite. The essential point for us is not to memorize every hardware topology, but to understand that **parallel computing often assumes a much tighter integration between processors** than a typical distributed system.

🔄 **Different primary objectives.** There is also a useful conceptual distinction in **why** we use multiple processors. In **parallel computing**, the central goal is often to **speed up one computation**. Suppose a computation takes time T on one processor, ideally with p processors we would like:

$$T_p \approx \frac{T}{p}$$

Although real systems obviously have overhead. Instead, **distributed computing** has broader goals: **scalability, geographical distribution, resource sharing, reliability, inherent distribution of users/data**. Performance can certainly be one objective, but it is **not the defining one**.

- Parallel computing: mainly cooperation for computational performance.
- Distributed computing: cooperation among autonomous networked components.

📌 Key Takeaways 4

- **A distributed system is made of independent computers/components that cooperate to provide one coherent**

system.

many independent machines $\rightarrow$ one logical service

- **Communication and coordination happen through message passing.** Components on different machines cannot simply access each other's memory:

$$P_1 \xrightarrow{\text{message}} P_2$$

- **Independence implies independent failures.** One machine may fail while the others continue running. This possibility of **partial failure** is one of the defining difficulties of distributed systems.
- **Distributed computing is not the same as parallel computing.** Both use multiple processors, but parallel systems are generally more tightly coupled and may use shared memory, whereas distributed systems consist of autonomous machines with local/private state connected through a network.
- **The main objective differs.** Parallel computing is primarily concerned with accelerating computation; distributed computing is broader and includes cooperation, scalability, geographical distribution, resource sharing, and reliability.

A distributed system consists of autonomous networked components that coordinate by message passing while appearing as one coherent system.

1.3 Middleware

Middleware is the software layer between the operating system/network and the distributed application.

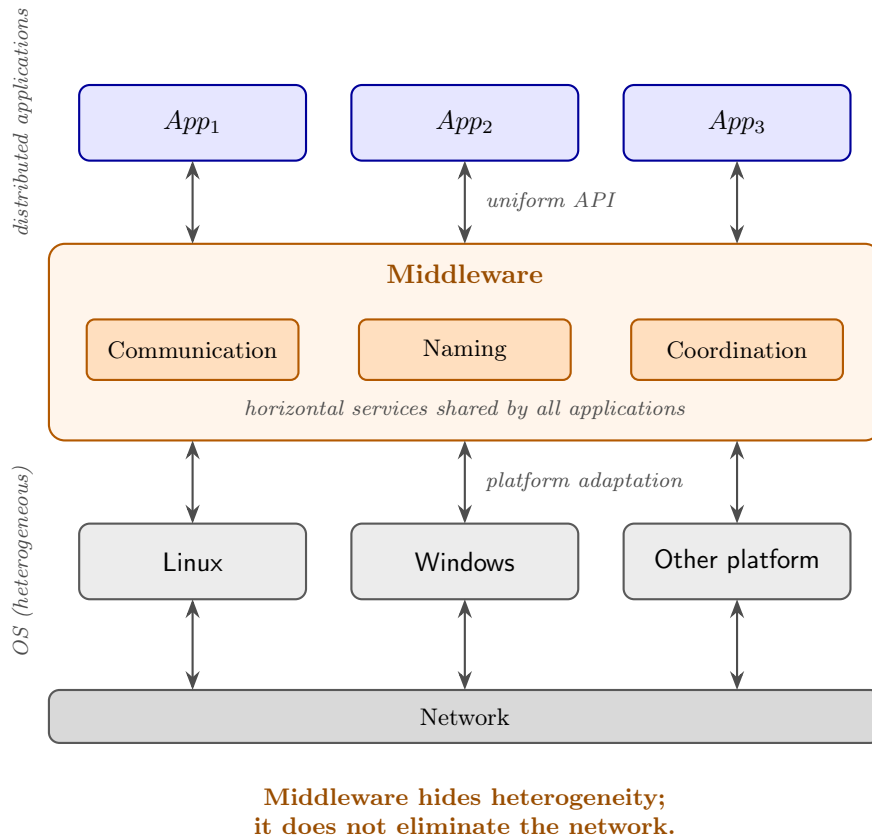


Figure 1: Middleware as a common layer between distributed applications and heterogeneous platforms.

The key idea is that applications should not have to deal directly with every low-level difference between machines, operating systems, and network mechanisms.

❓ Why middleware exists

Suppose we have two applications running on different machines: *A* and *B*. Machine *A* could use one operating system, while *B* uses another. They may expose different APIs, use different data representations, or provide different networking facilities.

- **Without middleware**, the application itself has to handle many of these differences:

Application → OS/network-specific details

This makes distributed applications harder to build and less portable.

- **Middleware** introduces an intermediate abstraction:

Application $\rightarrow$ Middleware API $\rightarrow$ OS/network

So that the application can interact with a **more uniform interface**.

The central purpose is to **hide some platform heterogeneity from distributed applications**.

⚠ Network/OS-based distributed system

One possibility is to **build the distributed application directly on top of the networking and operating-system facilities**. In this case, the application may need to deal directly with things such as: sockets, network addresses, data encoding, communication details, and differences between platforms. This gives the programmer **a lot of control**, but also exposes **a lot of complexity**.

✓ Middleware-based distributed system

With middleware, we introduce another layer between the application and the operating system/network. The **application uses the services provided by middleware** rather than directly implementing everything using low-level network primitives. This has **two main purposes**:

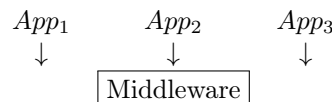
1. **Providing horizontal services**. Middleware provides **horizontal services** useful to many different distributed applications. “*Horizontal*” means that the **service is not specific to one particular business application**. For example, conceptually, different applications may all need common mechanisms such as communication, naming, and coordination, rather than implementing them independently every time.

So instead of:

$App_1 \rightarrow$ implement everything

$App_2 \rightarrow$ implement everything again

we obtain:



The **common distributed functionality is reusable**.

2. **Masking platform differences**. This is the second explicit role of middleware. Imagine a linux machine, a windows machine, and another platform (e.g., MacOS, Android, etc.). At the lower level, these systems may be quite **different**. **Middleware tries to offer the applications a common abstraction**, while internally adapting to each platform. So middleware acts somewhat like an **adapter layer**.

Notice that it does not magically make the underlying machines identical. It simply **masks some of the differences** so that application developers do not have to handle all of them directly.

Example 4: Middleware hides complexity

Suppose application A wants to invoke functionality provided by application B .

✗ Without a middleware abstraction, A might have to think in terms of:

IP address + socket + message format + network communication

✓ With middleware, the application can instead work with a higher-level operation:

$$A \rightarrow \boxed{\text{middleware service}} \rightarrow B$$

The low-level communication still happens, of course. Middleware does **not remove the network**, it just **hides part of its complexity behind a higher-level abstraction**.

Key Takeaways 5

Just three key points to remember about middleware:

1. Middleware sits between applications and the OS/network.
2. It provides reusable horizontal services for distributed applications.
3. It masks platform differences, making applications easier to build.

So in one sentence: **Middleware is a common software layer that simplifies distributed programming by providing shared services and hiding platform heterogeneity.**

1.4 Fundamental Properties

In this section we discuss three fundamental properties of distributed systems. More precisely, these are three characteristics that arise from the fact that a distributed system consists of **independent computers communicating over a network**: concurrency, no global clock, independent/partial failures. These properties are consequences of distribution itself.

✂ Concurrency

In a centralized system, concurrency is often a **design choice**. We may decide to write a single-threaded program, or a multi-threaded program. In a distributed system, instead, concurrency is essentially **unavoidable**. Suppose we have three independent computers: C_1 , C_2 , and C_3 . Each has its own processor and executes independently:

$$C_1 : a_1 \rightarrow a_2 \rightarrow a_3$$

$$C_2 : b_1 \rightarrow b_2 \rightarrow b_3$$

$$C_3 : c_1 \rightarrow c_2 \rightarrow c_3$$

Their computations are happening **at the same time**. Therefore, **concurrency is inherent in distributed systems**. This creates **coordination problems** whenever several components access or modify logically shared resources, like **synchronization**, **mutual exclusion**, and **consistency**.

🕒 Absence of a Global Clock

On one computer, we have a physical clock that the operating system can use as a common time reference. Different processes can therefore consult approximately the same underlying clock.

In a distributed system, however, each computer has its own physical clock:

$$C_1 \rightarrow \text{clock}_1$$

$$C_2 \rightarrow \text{clock}_2$$

$$C_3 \rightarrow \text{clock}_3$$

And these clocks are **not necessarily synchronized**:

$$\text{clock}_1 \neq \text{clock}_2 \neq \text{clock}_3$$

Even if we initially synchronize them, physical clocks drift. So there is **no perfectly shared global clock available to every component**.

This becomes especially problematic when we want to determine the **ordering of events**. Looking only at the local timestamps may not always be sufficient, because the clocks can disagree. So distributed systems need other ways to reason about ordering and synchronization. Later in this notes we will discuss **logical clocks** rather than physical clocks. However, in general, **there is no perfectly shared notion of time across all machines**.

⚠ Independent/Partial Failures

In a centralized system, if the machine crashes, typically the whole application fails. The failure is relatively easy to conceptualize.

In a distributed system, instead, each machine can fail independently:

$$C_1 : \checkmark \quad C_2 : \checkmark \quad C_3 : \times \quad C_4 : \checkmark$$

One component fails while the others continue running. As anticipated in the previous section, this is called a **partial failure**.

Also, **communication makes failure detection more difficult**. Suppose:

$$A \xrightarrow{\text{request}} B$$

And A receives no response. What happened?

1. B crashed
2. The request was lost
3. The reply was lost
4. B is simply slow
5. The network is extremely slow or unavailable

For A , all of these can initially appear as no response from B . Therefore determining whether another component has actually failed can be **hard or even impossible with certainty** under some assumptions. This is a much deeper problem than in an ordinary centralized application.

Finally, **recovery is also harder**. Suppose machine C_2 crashes and later restarts. *Where is the application state?* It may be distributed across C_1 , C_2 , and C_3 . Perhaps C_1 believes one thing while C_3 has slightly different information. So recovery is not simply a matter of restarting the crashed machine. We may need to reconstruct a **consistent distributed state**. This is why fault tolerance becomes such a large and important topic in distributed systems.

📌 Key Takeaways 6

Distributed systems inherently execute **concurrently**, have **no perfectly shared global clock**, and can suffer independent partial failures. These three properties are fundamental because **most of the difficult problems in distributed systems arise from them**.

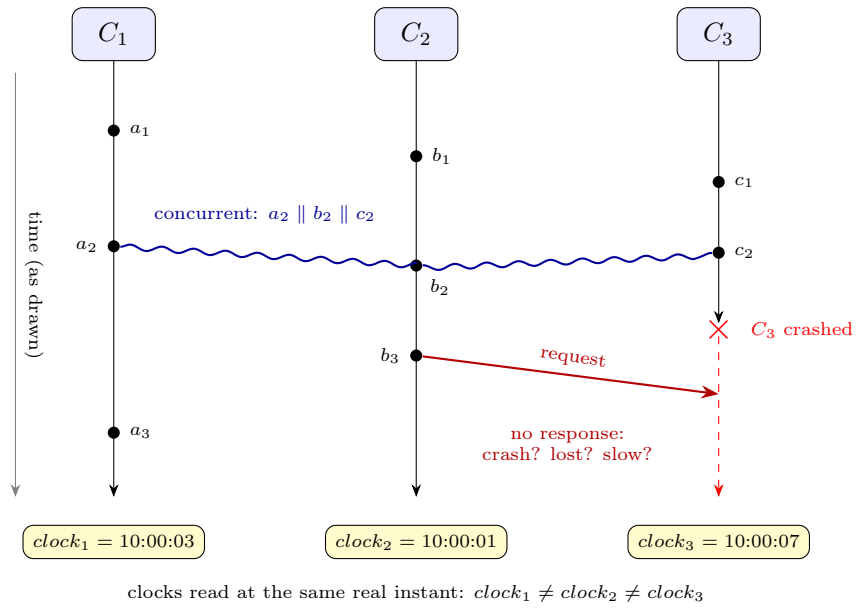


Figure 2: **The three fundamental properties of distributed systems.** Independent computers C_1, C_2, C_3 execute **concurrently** (blue wavy line: no causal path between the events), each has its own physical clock with **no global time** (yellow boxes), and a component such as C_3 can **fail while the others keep running**, leaving the sender unable to tell a crash from a lost message or a slow node (red).

References

- [1] Margara Alessandro Cugola Gianpaolo Saverio. Distributed systems. Slides from the HPC-E master's degree course on Politecnico di Milano, 2026.

Index

D

Distributed System 23–25

E

Eight Fallacies of Distributed Computing 19

M

Middleware 28

P

Pervasive Computing 8

U

Ubiquitous Computing 8